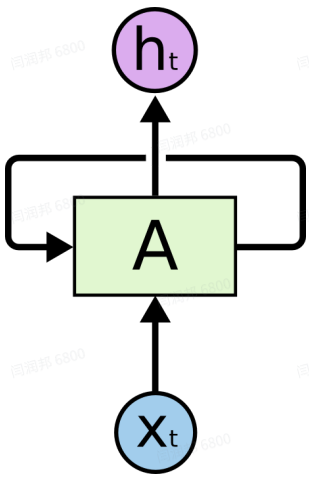
Modeling Programs Hierarchically with Stack-Augmented LSTM

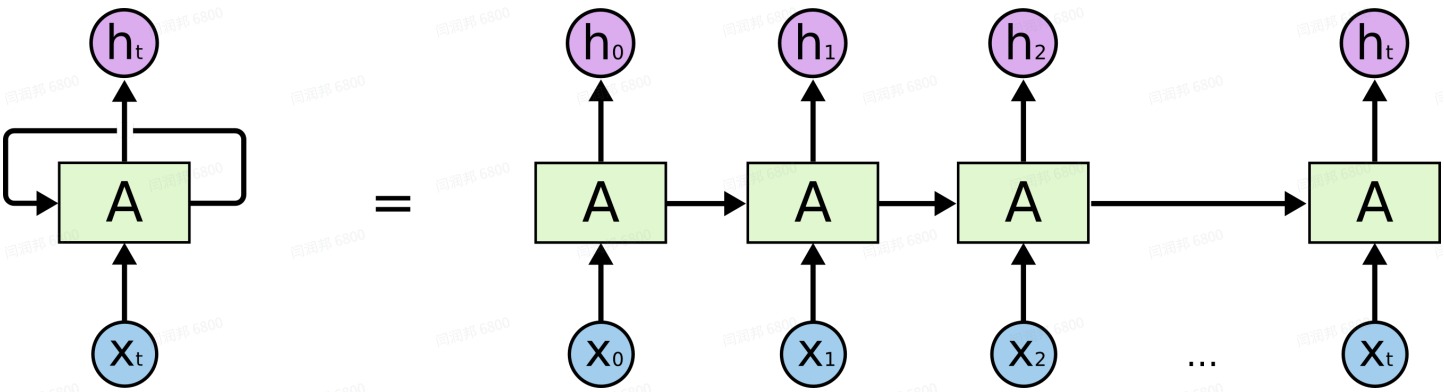
LSTM

Long Short Term Memory Networks，是一种 RNN （Recurrent Neural Network）。

循环神经网络（RNN）



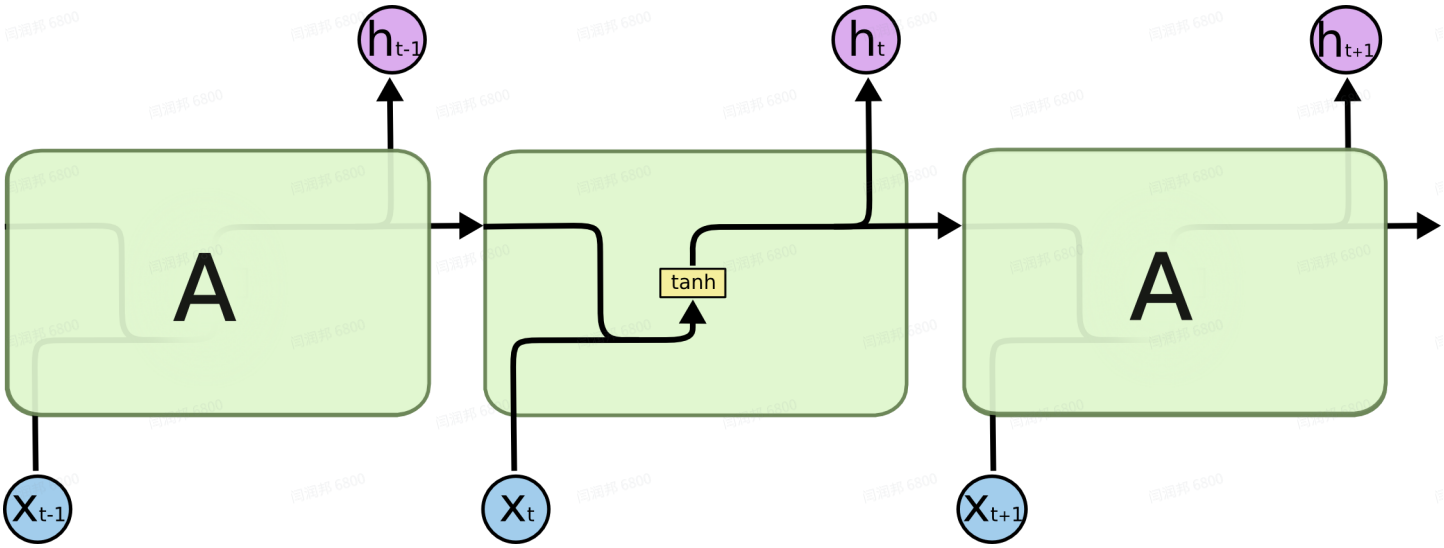
把这个RNN展开：



读入*上一个时刻的输出(y_{t-1})*和*这一时刻的输入(x_t)*，输出得到这一时刻的*隐藏层(h_t)*和*输出(y_t)*。这一时刻的输出会作为下一个时刻的输入

RNN可以用来进行语义理解、文本生成：RNN的输出表示对前文信息的理解和总结，可以根据前一时刻的输出，预测接下来的文本。

标准RNN的内部A：

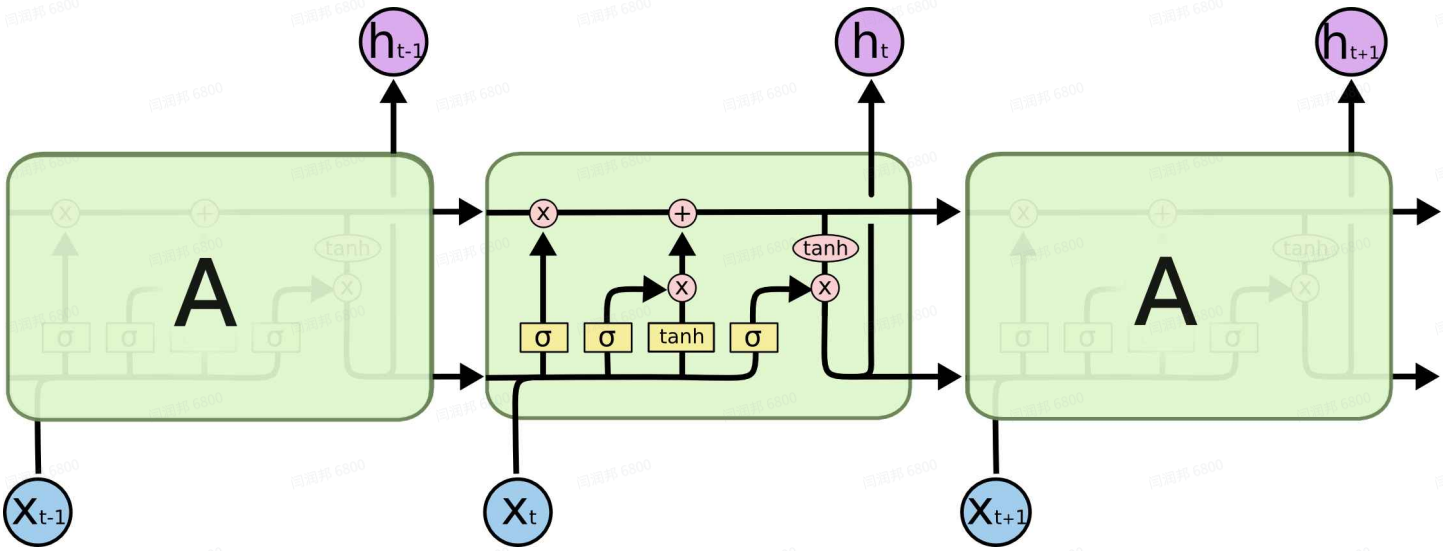


只有一个tanh激活层，即先对输入进行线性变换，然后经过tanh激活，得到隐藏层和输出。

循环神经网络的缺点：

难以捕捉长期的记忆依赖。比如：“I grew up in France…… I speak fluent *French*.”当需要预测*French*这一单词时，他所依赖的词*France*离它很远，这样随着RNN对新输入文本的记忆，France会被逐渐忘记。

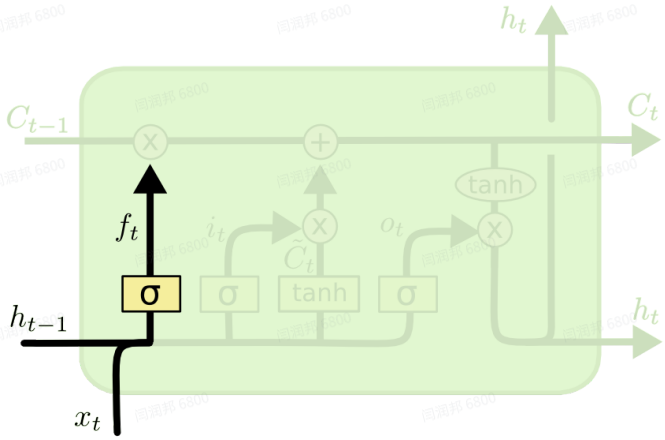
长短期记忆网络（LSTM）



更改了标准RNN的内部结构，但是整体流程没变，还是读取输入和上一时刻的输出，得到这一时刻的隐藏层和这一时刻的输出。

输出用 C_t 表示。

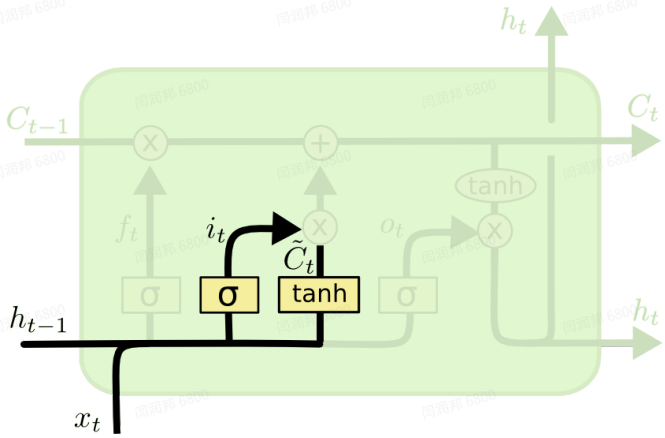
Forget Gate



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \ + \ b_f\right)$$

根据 h_{t-1} 和 x_t 计算出 f_t （forget），表示对于之前的记忆需要忘掉多少。由于使用的是sigmoid激活函数， $f_t \in (0,1)$ 。将 C_{t-1} 和 f_t 相乘，选择性地舍弃不重要的记忆。

Input Gate



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \ + \ b_i\right)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \ + \ b_C)$$

根据 h_{t-1} 和 x_t 计算出 i_t （input），表示对于当前输入，需要加入记忆的内容；

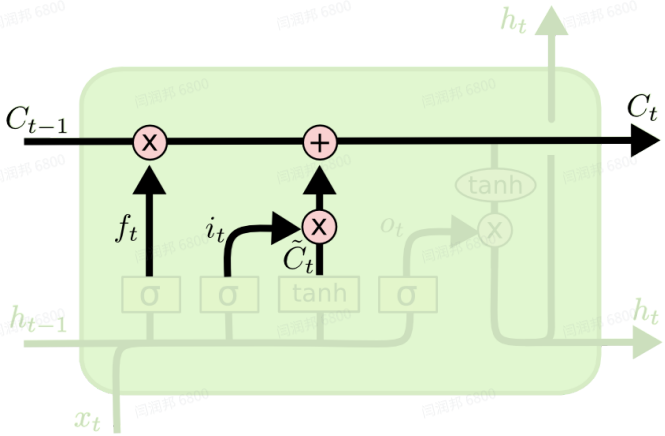
计算出

$$\tilde{C}_t$$

表示新加入的内容以什么比例加入，与 i_t 相乘，即选择性加入新记忆。

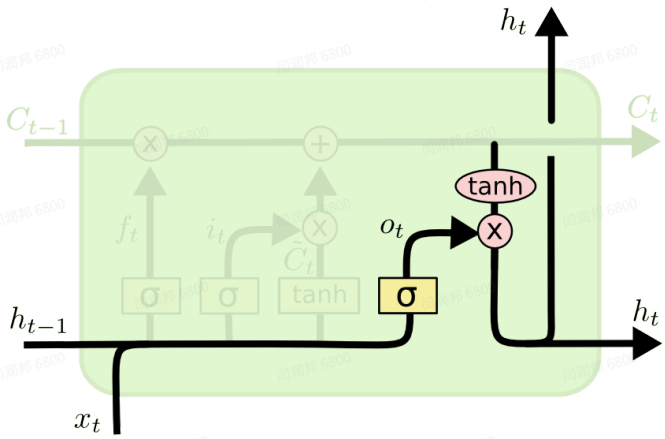
得到 C_t

经过Forget Gate和Input Gate后，得到了遗忘不重要内容的旧记忆和需要加入的新记忆，直接将这两部分相加，得到新的记忆。



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Output Gate



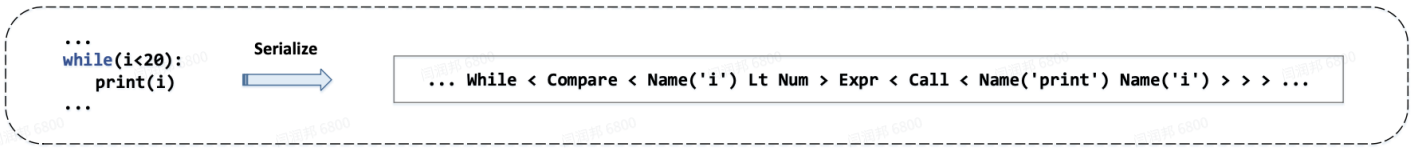
$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

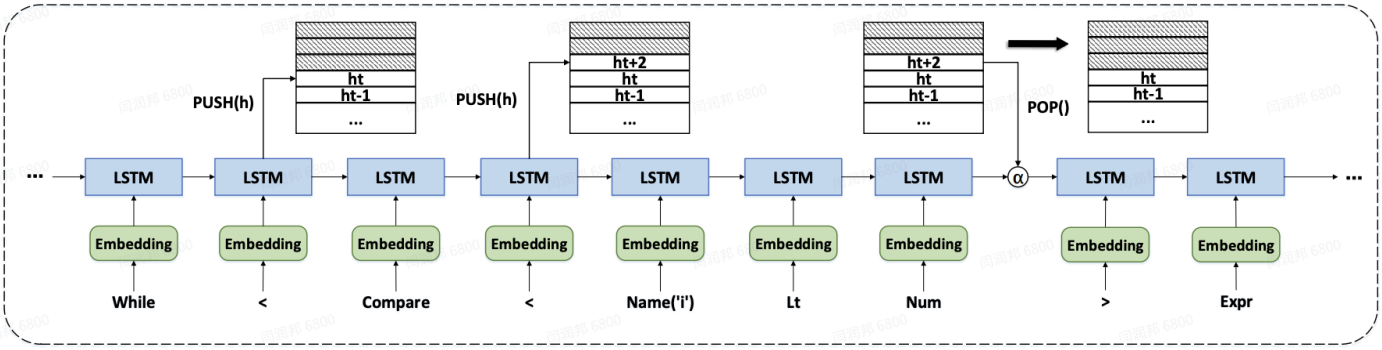
Stack-Augmented LSTM

普通的LSTM在处理自然语言时有很好的效果，但是处理程序语言的效果并不是很好。主要原因之一是：程序语言有固定的语法结构，LSTM并不能获取到程序语言的语法结构知识。

Stack-Augmented LSTM对程序的抽象语法树（Abstract Syntax Tree，AST）进行序列化，得到带有结构信息的代码表示，再将栈和LSTM结合起来，实现对程序语言的处理。



用<>把AST上一个节点的子节点包围起来。在LSTM中，每读取到一个“<”，说明LSTM需要开始理解新的代码块，于是把当前的隐藏层压入栈中，作为理解新代码块之前的状态保存起来；每读取到一个“>”，说明LSTM已经理解了内层的代码块，这时需要将栈顶元素弹出，结合理解该代码块之前的状态和对该代码块的理解，作为对整段代码的理解。



$$h_{context} = \begin{cases} h_{t-1},\ push(h_{t-1}) & x_t = V_{start} \\ \alpha(pop(), h_{t-1}) & x_t = V_{end} \\ h_{t-1} & Other\ cases \end{cases}$$

$$f_t = \sigma_g(W_f x_t + U_f h_{context} + b_f)$$

$$i_t = \sigma_g(W_i x_t + U_i h_{context} + b_i)$$

$$o_t = \sigma_g(W_o x_t + U_o h_{context} + b_o)$$

$$c_t = f_t \cdot c_{t-1} + i_t \cdot \tanh(W_c x_t + U_c h_{context} + b_c)$$

$$h_t = o_t \cdot \tanh c_t$$

INPUT: x

Initialize model parameters
for t=1 to numsteps **do**
  if $x_t = V_{start}$ **then**
    $PUSH(h_{t-1})$;
    $h_t \leftarrow lstm(x_t, h_{t-1})$
  else if $x_t = V_{end}$ **then**
    $h_{begin} \leftarrow POP()$
    $h_{context} \leftarrow \alpha(h_{begin}, h_{t-1})$
    $h_t \leftarrow lstm(x_t, h_{context})$
  else
    $h_t \leftarrow lstm(x_t, h_{t-1})$
  end if
end for

参考资料：

https://arxiv.org/pdf/2002.04516.pdf

Understanding LSTM Networks -- colah's blog